BORDERLESS
<CODING>

Rust for Web3 development

Part 04 · PDA + Project Prep

PDAs for deterministic state

Solana CLI for diagnostics

Final project architecture readiness

Learning outcomes

- Understand what PDAs are and why they matter
- Derive PDA addresses and reason about seeds/bump
- Use Solana CLI for practical debugging and verification
- Prepare architecture choices for final projects

Why this part exists

- Part 01: hello flow
- Part 02: Anchor core wiring
- Part 03: state lifecycle
- **Part 04: deterministic state addressing + project design**

PDA in one sentence

```
A PDA is a deterministic program-owned address derived from seeds + program ID.
```

PDA mental model

- `wallet pubkey` = user-owned keypair
- PDA = **no private key**, derived address
- program can "sign" for PDA via seeds + bump (`invoke_signed`)
- perfect for canonical on-chain state slots

Why use PDA instead of random keypair account

- deterministic lookup (same inputs => same address)
- no need to pass/store extra random keypairs
- cleaner authority and ownership patterns
- easier indexing and project-level architecture

PDA advantages (highlight)

- **Determinism**: predictable address from business keys
- **Security model**: program controls PDA writes
- **Developer UX**: simpler client account discovery
- **Project scaling**: seed schemas become architecture

Seeds + bump

- **Seeds**: stable bytes used in derivation
- **Bump**: extra byte chosen so address is off-curve
- Derivation API searches bump that produces valid PDA
- Store bump when needed for future signed CPI patterns

Common seed patterns

- `["profile", user_pubkey]`
- `["vault", market_pubkey]`
- `["bid", auction_pubkey, bidder_pubkey]`
- `["config"]` (singleton account)

Anchor PDA constraints (core idea)

```
#[account(
  init,
  payer = signer,
  space = 8 + ...,
  seeds = [b"profile", signer.key().as_ref()],
  bump
)]
pub profile: Account<'info, Profile>,
```

Counter with PDA (minimal example)

```
#[account]
pub struct Counter { pub value: u64 }

#[account(
  init,
  payer = signer,
  space = 8 + 8,
  seeds = [b"counter", signer.key().as_ref()],
  bump
)]
pub counter: Account<'info, Counter>,
```

Counter instructions (brief)

```
pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
  ctx.accounts.counter.value = 0;
  Ok(())
}

pub fn increment(ctx: Context<Increment>) -> Result<()> {
  ctx.accounts.counter.value += 1;
  msg!("counter = {}", ctx.accounts.counter.value);
  Ok(())
}
```

PDA pitfalls to avoid

- changing seed schema mid-project
- using non-stable seeds (e.g. mutable strings)
- wrong account passed from client (`ConstraintSeeds`)
- forgetting space growth plan for future fields

Solana CLI: minimum command set

- `solana config get`
- `solana address`
- `solana balance`
- `solana airdrop 1`
- `solana account <PUBKEY>`
- `solana logs`

Anchor + CLI workflow (practical)

1. `anchor build`
2. `anchor test` or deploy flow
3. verify account data with `solana account <pubkey>`
4. inspect runtime errors via `solana logs`

CLI checks before debugging code

- ▪ correct cluster (`devnet` / localnet)
- ▪ correct wallet keypair
- ▪ enough SOL for payer
- ▪ program id alignment (`declare_id!` vs deployed ID)

PDA-first project thinking

- Define core entities first (`Auction`, `Bid`, `UserProfile`, `CollectionConfig`)
- Decide canonical PDA for each entity
- Define who can mutate each account
- Define read paths for client queries

Final project prep: architecture checklist

- account model diagram
- seed schema per account
- authority model per instruction
- invariants (what must always be true)
- test matrix (happy + failure paths)

NFT project mapping (example)

- `CollectionConfig` PDA: `["config"]`
- `MintState` PDA: `["mint_state", mint_pubkey]`
- optional user profile PDA: `["profile", user_pubkey]`
- authority checks around mint/update controls

Auction project mapping (example)

- `Auction` PDA: `["auction", creator_pubkey, auction_id]`
- `Bid` PDA: `["bid", auction_pubkey, bidder_pubkey]`
- `Vault` PDA for escrow patterns
- explicit close/settle lifecycle rules

Theory to implementation bridge

- PDAs solve **addressing**
- constraints solve **safety**
- tests solve **confidence**
- CLI solves **runtime visibility**

What to have ready before Part 05

- clear PDA naming scheme
- one page account schema
- instruction list with authorities
- baseline tests template

Part 04 recap

- PDA = deterministic program-owned address
- seeds+bump are design, not syntax trivia
- CLI is essential for fast diagnosis
- good project outcomes start with account modeling